<?php

UNIDAD DIDÁCTICA 02

Tutorial 03
Acceso de usuarios

Formularios, autenticación y sesiones PHP

PHP · Sesiones · Login · Seguridad

UD 02 · DWES · PHP 8.1+

Tutorial 03 · Acceso de usuarios con sesiones

Objetivo: Vamos a construir un acceso a un club de lectura. Una persona introduce correo y contraseña en un formulario; si los datos son correctos, entra en una página privada. La sesión conserva la identidad entre solicitudes y el botón de salida la elimina.

Antes de comenzar

Utiliza PHP 8.1 o posterior y un servidor web local (por ejemplo, Apache de XAMPP). Crea una carpeta de proyecto dentro de `htdocs/UD2/` y **escribe los archivos con el contenido mostrado en cada paso**. El directorio `Codigo/` contiene esos mismos archivos listos para ejecutar.

Crea la carpeta `htdocs/UD2/Login/`. Los archivos estarán todos al mismo nivel:

```
Login/
├── usuarios.php
├── login.php
├── zona.php
└── logout.php
```

La primera página será `http://localhost/UD2/Login/login.php`. No hay base de datos: las cuentas de demostración se definen en un archivo PHP.

Paso 1. Preparar las cuentas y la función de autenticación

Crea `usuarios.php`. La función `obtenerUsuariosDemo()` devuelve un array asociativo cuya clave es el correo electrónico. Cada elemento contiene un nombre y un hash de contraseña generado con `password_hash()`.

La función `autenticar($email, $clave)` convierte el correo a minúsculas, busca la cuenta y comprueba la contraseña con `password_verify()`. Si todo es correcto devuelve el nombre; si falla devuelve `null`. No genera mensajes HTML: se encarga solamente de comprobar datos.

Las credenciales para reproducir esta demostración local son `ana@club.test` / `claveAna123` y `luis@club.test` / `claveLuis123`. Los hashes se generan al cargar el archivo solo para simplificar el ejemplo; con base de datos se guardarían una vez al registrar la cuenta.

Archivo `Codigo/usuarios.php`

```php
<?php
declare(strict_types=1);

function obtenerUsuariosDemo(): array
{
    // Solo para aprendizaje local: en una aplicación real los hashes están almacenados.
    return [
        'ana@club.test' => [
            'nombre' => 'Ana',
            'hash' => password_hash('claveAna123', PASSWORD_DEFAULT)
        ],
        'luis@club.test' => [
            'nombre' => 'Luis',
            'hash' => password_hash('claveLuis123', PASSWORD_DEFAULT)
        ]
    ];
```

```
}

function autenticar(string $email, string $clave): ?string
{
    $cuentas = obtenerUsuariosDemo();
    $email = strtolower(trim($email));
    if (!isset($cuentas[$email])) {
        return null;
    }
    if (!password_verify($clave, $cuentas[$email]['hash'])) {
        return null;
    }
    return $cuentas[$email]['nombre'];
}
```

Un hash no es una contraseña cifrada que deba descifrarse: `password_verify()` comprueba si la contraseña introducida corresponde al hash guardado. Nunca guardamos la contraseña en una cookie.

Paso 2. Crear el formulario y procesar el acceso

Crea `login.php`. Antes de emitir HTML se ejecuta `session_start()` para abrir la sesión y se importa `usuarios.php`. Si ya existe `$_SESSION['nombre']`, se redirige a `zona.php`.

Cuando el formulario envía `POST`, el servidor recupera `email` y `clave`, llama a `autenticar()` y recibe un nombre o `null`. Si el nombre existe, `session_regenerate_id(true)` cambia el identificador de sesión y se almacena el nombre en `$_SESSION`. A continuación `header('Location: zona.php')` redirige y `exit` termina la respuesta. Si el acceso falla, se muestra un error general.

El HTML situado después de `?>` contiene los campos del formulario. El atributo `name` de cada `input` determina qué clave aparecerá en `$_POST`.

Archivo `Codigo/login.php`

```
<?php
declare(strict_types=1);
session_start();
require_once __DIR__ . '/usuarios.php';

if (isset($_SESSION['nombre'])) {
    header('Location: zona.php');
    exit;
}

$error = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email = trim((string) ($_POST['email'] ?? ''));
    $clave = (string) ($_POST['clave'] ?? '');
    $nombre = autenticar($email, $clave);

    if ($nombre !== null) {
        session_regenerate_id(true);
        $_SESSION['nombre'] = $nombre;
        header('Location: zona.php');
        exit;
    }
    $error = 'Credenciales incorrectas.';
}
?>
<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Acceso</title>
```

```
</head>
<body>
    <h1>Club de lectura</h1>
    <?php if ($error !== ''): ?>
        <p><?= htmlspecialchars($error, ENT_QUOTES, 'UTF-8') ?></p>
    <?php endif; ?>
    <form action="login.php" method="post">
        <label>Correo
            <input name="email" type="email" required>
        </label>
        <label>Contraseña
            <input name="clave" type="password" required>
        </label>
        <button type="submit">Entrar</button>
    </form>
</body>
</html>
```

Las redirecciones con `header()` requieren no haber enviado texto ni HTML previamente; por eso la parte PHP que decide el acceso aparece antes del `<!doctype html>`.

Paso 3. Proteger el contenido privado

Crea `zona.php`. Cada petición es independiente, por lo que debemos ejecutar otra vez `session_start()` al entrar. Si `$_SESSION['nombre']` no existe, se redirige a `login.php` sin presentar contenido protegido.

Si la sesión existe, el nombre pasa por `htmlspecialchars()` antes de insertarse en el título. El formulario de salida utiliza `method="post"` y apunta a `logout.php`.

Archivo `Codigo/zona.php`

```
<?php
declare(strict_types=1);
session_start();

if (!isset($_SESSION['nombre'])) {
    header('Location: login.php');
    exit;
}
$nombre = htmlspecialchars((string) $_SESSION['nombre'], ENT_QUOTES, 'UTF-8');
?>
<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Zona privada</title>
</head>
<body>
    <h1>Bienvenida, <?= $nombre ?></h1>
    <form action="logout.php" method="post">
        <button type="submit">Cerrar sesión</button>
    </form>
</body>
</html>
```

Es importante proteger **la propia página privada**, no solo ocultar o mostrar enlaces desde el login: alguien puede escribir `zona.php` directamente en la barra de direcciones.

Paso 4. Cerrar la sesión correctamente

Crea `logout.php`. El archivo rechaza peticiones que no sean `POST` con el código HTTP 405. Tras `session_start()`, vacía `$_SESSION`, solicita al navegador que elimine la cookie de sesión usando los mismos parámetros de la cookie original, destruye los datos de sesión mediante `session_destroy()` y redirige al formulario de acceso.

Aquí tampoco hay HTML porque este archivo solo procesa una acción y redirige.

Archivo `Codigo/logout.php`

```
<?php
declare(strict_types=1);

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    exit;
}

session_start();
$_SESSION = [];
if (ini_get('session.use_cookies')) {
    $parametros = session_get_cookie_params();
    setcookie(session_name(), '', [
        'expires' => time() - 3600,
        'path' => $parametros['path'],
        'domain' => $parametros['domain'],
        'secure' => $parametros['secure'],
        'httponly' => $parametros['httponly'],
        'samesite' => $parametros['samesite'] ?? 'Lax'
    ]);
}
session_destroy();
header('Location: login.php');
exit;
```

La cookie de PHP contiene normalmente un **identificador de sesión**, no el nombre ni la contraseña del usuario. La información asociada al identificador se gestiona en el servidor.

Resultado y comprobación

1. Abre `http://localhost/UD2/Login/zona.php` en una sesión nueva: serás redirigido a `login.php`.
2. Entra con `ana@club.test` y `claveAna123`: aparece «Bienvenida, Ana».
3. Recarga `zona.php`: el acceso se mantiene gracias a la sesión.
4. Pulsa «Cerrar sesión» y vuelve a intentar entrar directamente en `zona.php`: regresas al formulario de acceso.
5. Prueba una contraseña incorrecta: se muestra «Credenciales incorrectas» y no se accede a la zona privada.

Lo importante: `POST` transporta las credenciales, `password_verify()` las comprueba, `$_SESSION` mantiene el acceso y cada página privada debe comprobar la sesión. En un despliegue real hay que añadir HTTPS, protección CSRF y límites de intentos de acceso.